

```

64.     """Encrypt the message using the key."""
65.     return translateMessage(message, key, 'encrypt')
66.
67.
68. def decryptMessage(message, key):
69.     """Decrypt the message using the key."""
70.     return translateMessage(message, key, 'decrypt')
71.
72.
73. def translateMessage(message, key, mode):
74.     """Encrypt or decrypt the message using the key."""
75.     translated = [] # Stores the encrypted/decrypted message string.
76.
77.     keyIndex = 0
78.     key = key.upper()
79.
80.     for symbol in message: # Loop through each character in message.
81.         num = LETTERS.find(symbol.upper())
82.         if num != -1: # -1 means symbol.upper() was not in LETTERS.
83.             if mode == 'encrypt':
84.                 # Add if encrypting:
85.                 num += LETTERS.find(key[keyIndex])
86.             elif mode == 'decrypt':
87.                 # Subtract if decrypting:
88.                 num -= LETTERS.find(key[keyIndex])
89.
90.             num %= len(LETTERS) # Handle the potential wrap-around.
91.
92.             # Add the encrypted/decrypted symbol to translated.
93.             if symbol.isupper():
94.                 translated.append(LETTERS[num])
95.             elif symbol.islower():
96.                 translated.append(LETTERS[num].lower())
97.
98.             keyIndex += 1 # Move to the next letter in the key.
99.             if keyIndex == len(key):
100.                 keyIndex = 0
101.         else:
102.             # Just add the symbol without encrypting/decrypting:
103.             translated.append(symbol)
104.
105.     return ''.join(translated)
106.
107.
108. # If this program was run (instead of imported), run the program:
109. if __name__ == '__main__':
110.     main()

```
